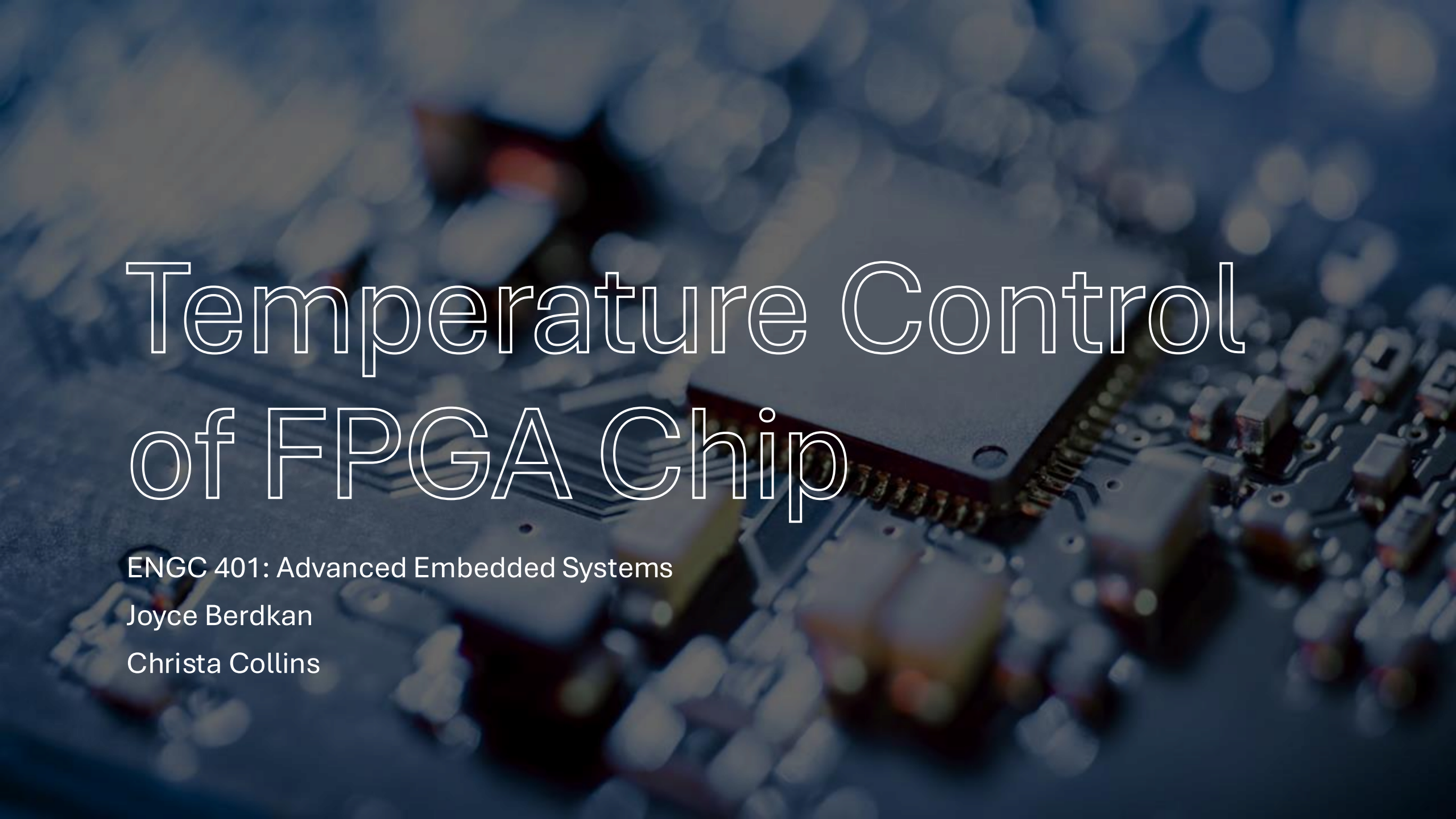


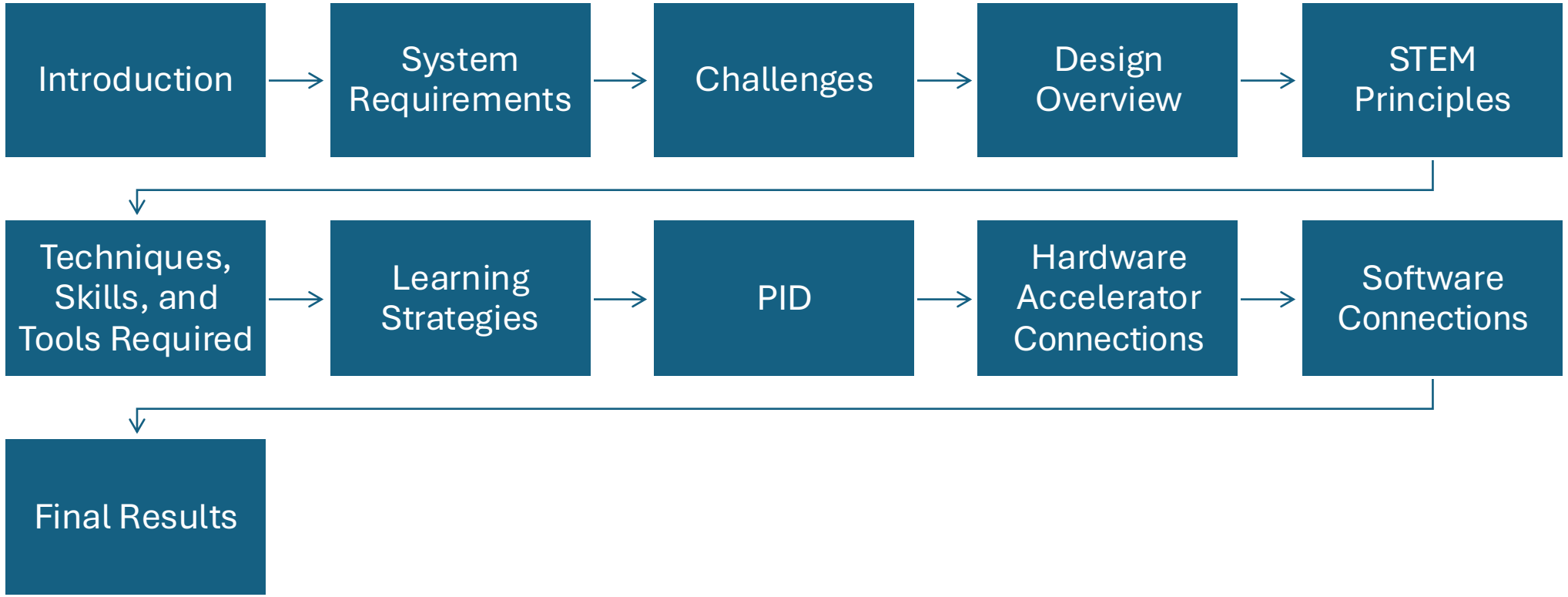
Temperature Control of FPGA Chip

ENGC 401: Advanced Embedded Systems

Joyce Berdkan

Christa Collins

Outline


Introduction
System Requirements
Challenges
Design Overview
STEM Principles
Techniques, Skills, and Tools Required
Learning Strategies
PID
Hardware Accelerator Connections
Software Connections
Final Results

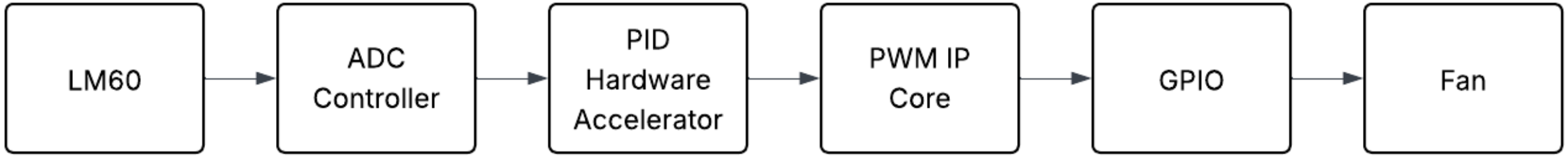
System Requirements

Requirement ID	Requirement Description
R.1	System must contain the following IP cores: ADC, PWM, and PID
R.2	Must design custom PID IP core wrapped in the Avalon bus
R.3	The PID IP cores must be implemented in the hardware
R.4	Build a circuit for driving a DC fan

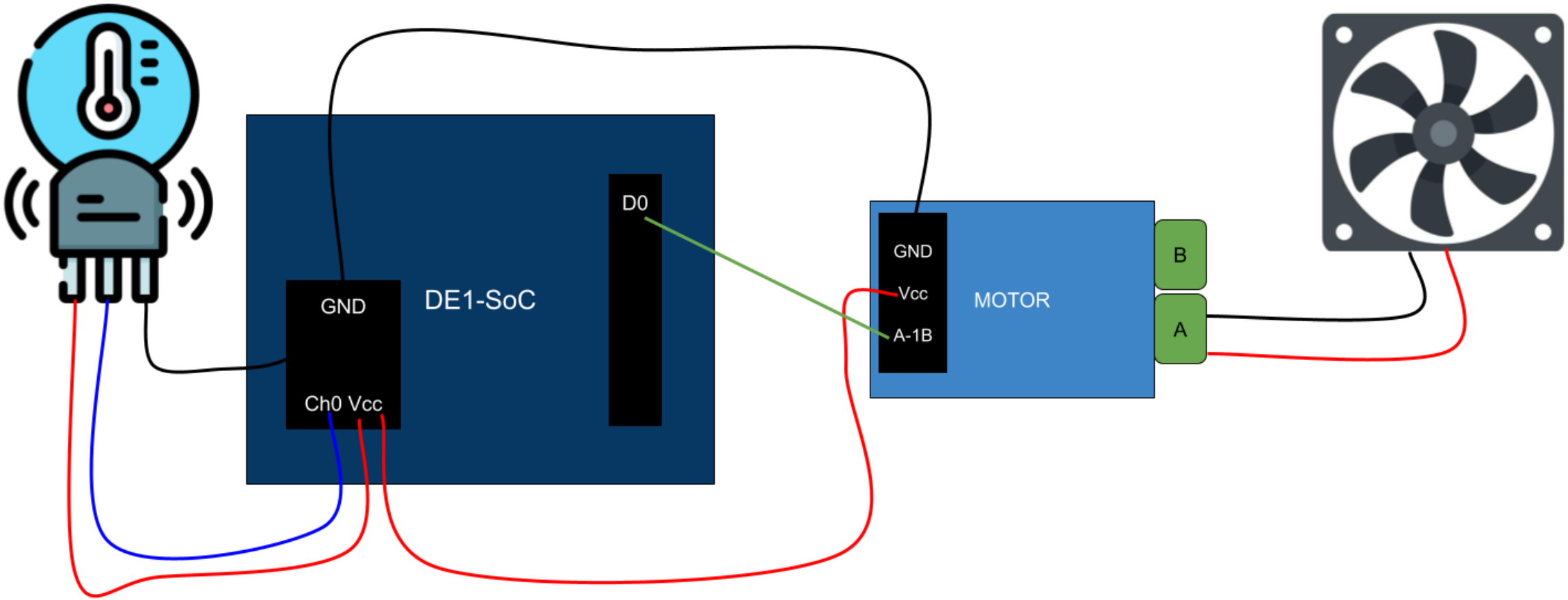
Challenges

- This project is challenging as it spans multiple domains:
  - Integrating principles from engineering, science, and math
  - Multi-step hardware integration
  - Complex embedded systems integration
  - Real-time control management
  - Ensuring clear communication between the hardware and software components

Design Overview: Computer System Design



Design Overview: Circuit Schematic



Fulfills requirement R.4: Build a circuit for driving a DC fan

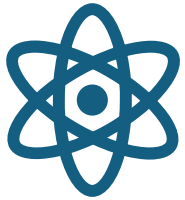
STEM Principles

Engineering Principles

Embedded systems design

Hardware acceleration

Control systems engineering

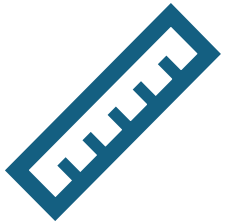


Science Principles

Thermal physics

Sensor technology

Signal processing



Math Principles

Proportional-Integral-Derivative (PID)

Techniques, Skills, and Tools Required

Techniques

- Implementing PID on FPGA
- System integration
- ADC Data Processing

Skills

- Embedded systems design
- Signal processing
- Mathematics

Tools

- DE1-SoC (FPGA)
- Quartus (Qsys)
- LM60 (temperature sensor)
- L9110 (motor driver)

Learning Strategies

- Reviewed papers detailing theory and hardware implementation of PID control in FPGA
- Analyzed GitHub PID VHDL code to understand modular design, state machines, and signal processing
- Translated the PID control logic into an Algorithmic State Machine Diagram (ASMD)

Learning Strategies

- Modeled proportional, integral and derivative calculations in hardware
- Regularly tested PID output on the DE1-SoC board using the LM60 temperature sensor
- Used Lab 5 as a guide for integrating the PID algorithm as a hardware accelerator via a custom IP core and Avalon-MM interface

PID Formula: Continuous Formula

- PID controllers minimize the difference between a desired setpoint r(t) and the measured output y(t) of a system.
- The error $e(t) = r(t) - y(t)$ is used to compute the control system $u(t)$:
- $u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$
- Where:
  - K_p: Proportional gain
  - K_i: Integral gain
  - K_d: Derivative gain

PID Formula: Discrete Formula

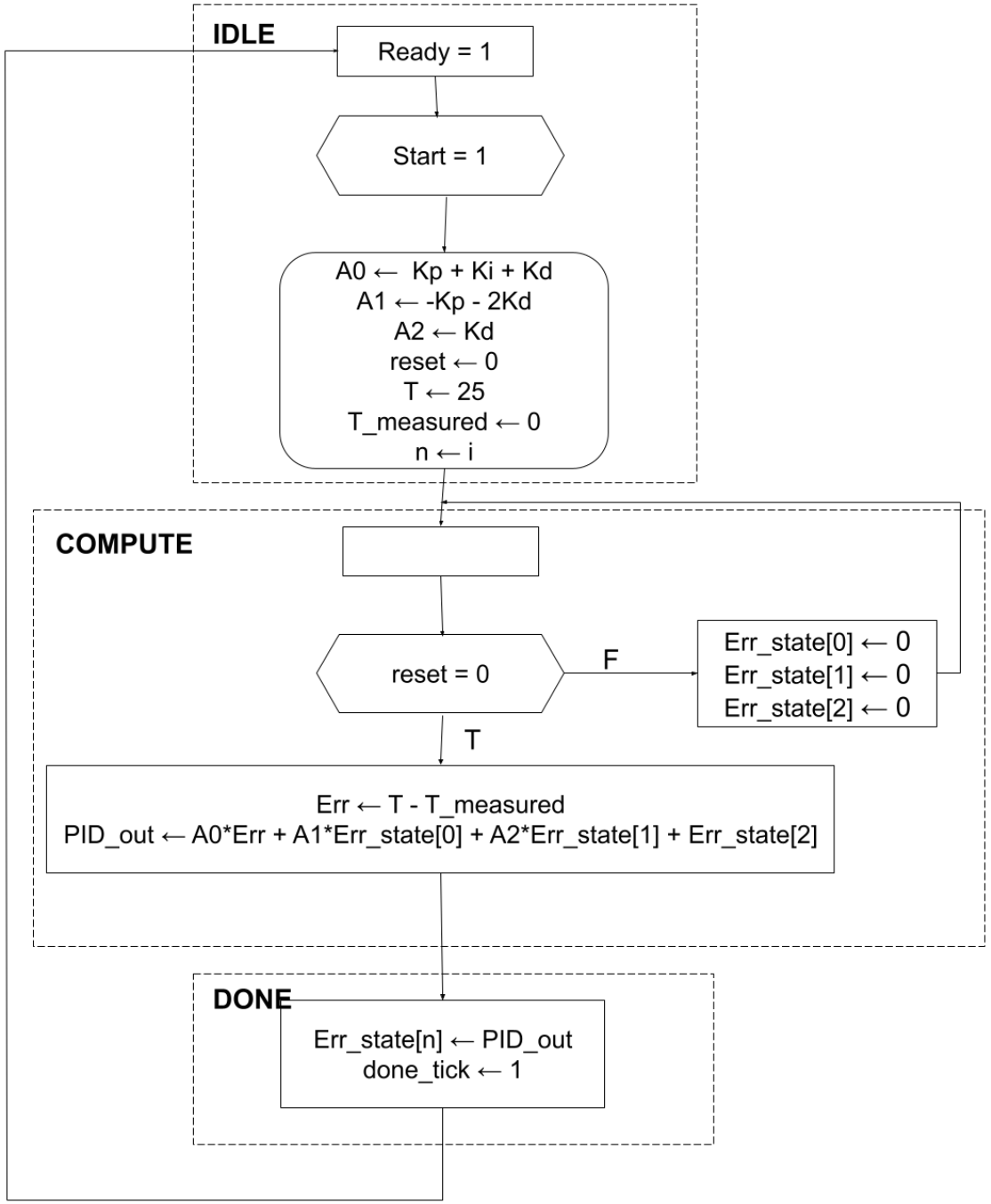
- By analyzing $u(t)$ in its frequency (Laplace) domain. We compute: $C(s) = K_p + \frac{K_i}{s} + K_d s$
- Through calculations we find its coefficients:
  - $A0 = K_p + K_i + K_d$
  - $A1 = -K_p - 2K_d$
  - $A2 = K_d$
- Such that the discrete formula of $u(t)$ can be described as:
  - $y(n) = y(n-1) + A0(x[n]) + A1(x[n-1]) + A2(x[n-2])$

PID Formula: Discrete Formula

$$y(n) = y(n-1) + A0(x[n]) + A1(x[n-1]) + A2(x[n-2])$$

- Wherein:
  - $y(n)$: current PID output
  - $y(n-1)$: previous PID output
  - $x[n]$: current error
  - $x[n-1]$: previous error
  - $x[n-2]$: error from two iterations ago

PID ASMD Chart



PID Feedback Loop

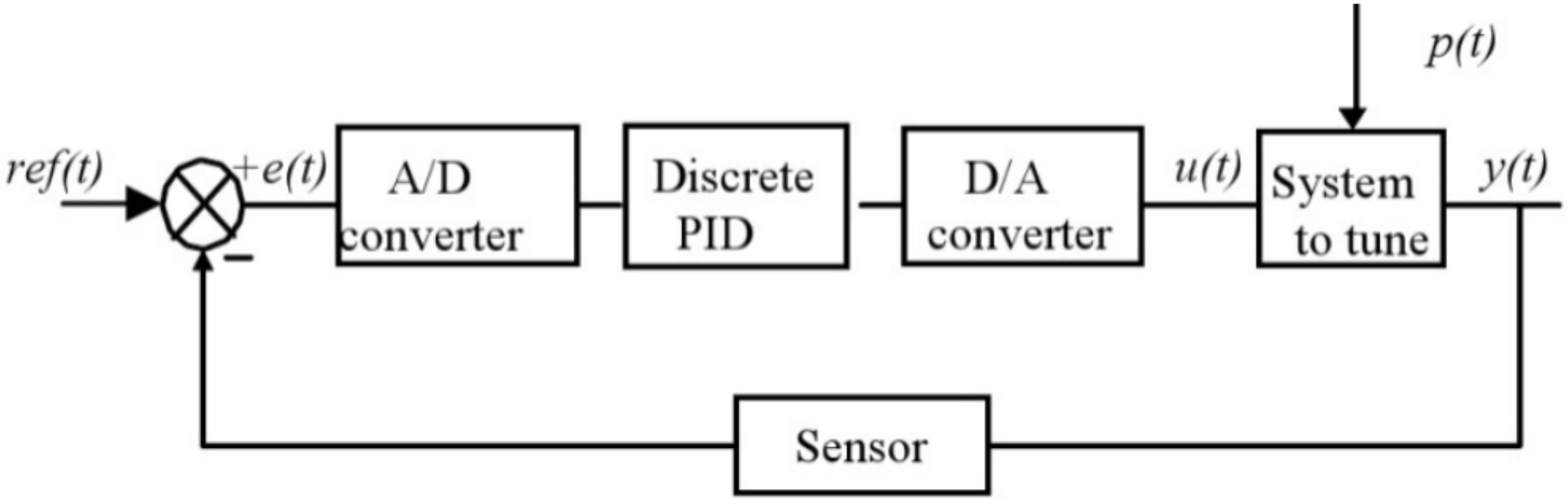


Figure retrieved from: https://www.intechopen.com/chapters/19194

PID FSMD

```
28   localparam [1:0]
29              IDLE      = 2'b00,
30              COMPUTE   = 2'b01,
31              DONE      = 2'b10;
32
33   //bdy
34   //FSMD state transition
35  always @(posedge clk or posedge reset) begin
36      if (reset) begin
37          state_reg <= IDLE;
38          kp_reg           <= 0;
39          ki_reg           <= 0;
40          kd_reg           <= 0;
41          setpoint_reg     <= 0;
42          meas_reg <= 0;
43          prev_error_reg   <= 0;
44          error_reg        <= 0;
45          derivative_reg   <= 0;
46          integral_reg     <= 0;
47          output_reg       <= 0;
48          pid_out_reg      <= 0;
49          done             <= 0;
50       end else begin
51       state_reg <= state_next;
52      case (state_reg)
53          IDLE: begin
54              done <= 0;
55                  if (start) begin
56                      setpoint_reg <= setpoint;
57                      meas_reg <= measurement;
```

```
49          done             <= 0;
50       end else begin
51       state_reg <= state_next;
52      case (state_reg)
53          IDLE: begin
54              done <= 0;
55                  if (start) begin
56                      setpoint_reg <= setpoint;
57                      meas_reg <= measurement;
58                      kp_reg <= kp;
59                      ki_reg <= ki;
60                      kd_reg <= kd;
61                  end
62              end
63              COMPUTE: begin
64                  error_reg <= setpoint_reg - meas_reg;
65                  integral_reg <= integral_reg + (setpoint_reg - meas_reg);
66                  derivative_reg <= (setpoint_reg - meas_reg) - prev_error_reg;
67                  prev_error_reg <= setpoint_reg - meas_reg;
68
69                  output_reg <= (kp_reg * error_reg) + ((ki_reg * integral_reg)      ) + (kd_reg * derivative_reg);
70
71              end
72              DONE: begin
73                  pid_out_reg <= output_reg[15:0];
74                  done <= 1;
75              end
76          endcase
77      end
78  end
```

PID Register Mapping

Address Map | System Contents | Parameters

System: soc_system **Path:** hps_0

	fpga_only_master.master	hps_0.h2f_axi_master	hps_0.h2f_lw_axi_master
adc.adc_slave			0x0000 4000 - 0x0000 401f
engc401_pid_0.pid			0x0000 0200 - 0x0000 021f
engc401_pwm_0.pwm			0x0000 0100 - 0x0000 010f
hps_0.f2h_axi_slave			
ntr_capturer_0.avalon_sl...	0x0003 0000 - 0x0003 0007		
tag_uart.avalon_jtag_slave	0x0002 0000 - 0x0002 0007		0x0002 0000 - 0x0002 0007
onchip_memory2_0.s1	0x0000 0000 - 0x0000 ffff	0x0000 0000 - 0x0000 ffff	
pio_0.s1			0x0000 0060 - 0x0000 007f
sysid_qsys.control_slave	0x0001 0000 - 0x0001 0007		0x0001 0000 - 0x0001 0007

Address Mapping

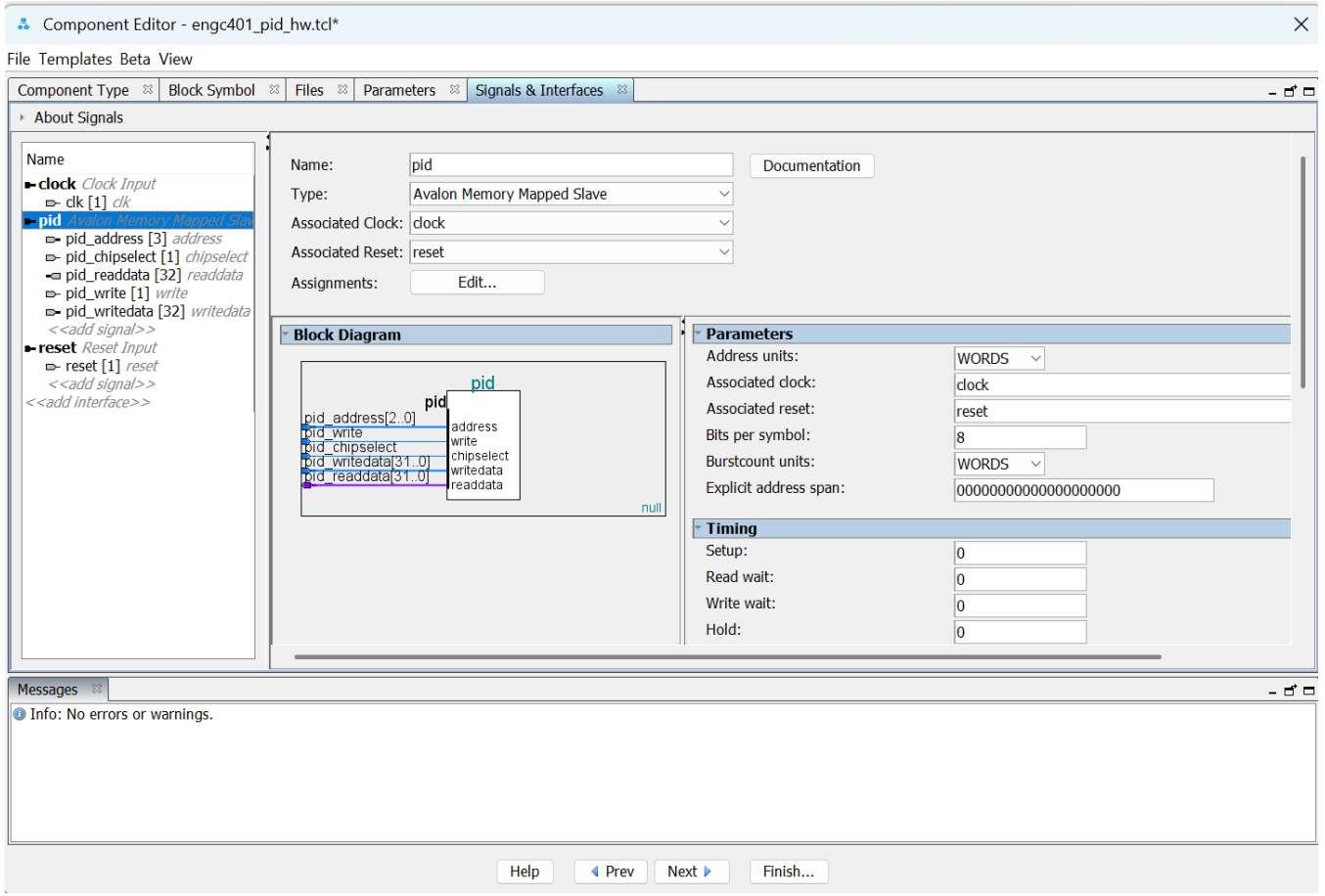
PID Wrapping Circuit

```
1  module avalon_pid
2  (
3      input wire clk, reset,
4      //Avalon-MM slave interface
5      input wire [2:0] pid_address,
6      input wire pid_write, pid_chipselect,
7      input wire [31:0] pid_writedata,
8      output wire [31:0] pid_readdata
9  );
10
11
12
13 //Signals
14 reg [15:0] setpoint_reg, meas_reg, kp_reg, ki_reg, kd_reg;
15 wire start;
16 wire ready, done;
17 wire [15:0] pid_out;
18
19 //PID engine instance
20 pid_engine core (
21     .clk(clk), .reset(reset), .start(start),
22     .setpoint(setpoint_reg), .measurement(meas_reg),
23     .kp(kp_reg), .ki(ki_reg), .kd(kd_reg),
24     .ready(ready), .done(done), .pid_out(pid_out)
25 );
26
27 //register write decoding
28 wire wr_en = pid_write & pid_chipselect;
29 wire wr_kp      = (pid_address == 3'b000) & wr_en;
30 wire wr_ki      = (pid_address == 3'b001) & wr_en;
```

```
28 wire wr_en = pid_write & pid_chipselect;
29 wire wr_kp      = (pid_address == 3'b000) & wr_en;
30 wire wr_ki      = (pid_address == 3'b001) & wr_en;
31 wire wr_kd      = (pid_address == 3'b010) & wr_en;
32 wire wr_set     = (pid_address == 3'b011) & wr_en;
33 wire wr_meas    = (pid_address == 3'b100) & wr_en;
34 wire wr_start   = (pid_address == 3'b101) & wr_en;
35
36
37 //Register updates
38 always @(posedge clk or posedge reset) begin
39     if (reset) begin
40         kp_reg      <= 0;
41         ki_reg      <= 0;
42         kd_reg      <= 0;
43         setpoint_reg <= 0;
44         meas_reg    <= 0;
45         //start <= 0;
46     end else begin
47         //start <=0;   //one-shot pulse
48         if (wr_kp)    kp_reg <= pid_writedata[15:0];
49         if (wr_ki)    ki_reg <= pid_writedata[15:0];
50         if (wr_kd)    kd_reg <= pid_writedata[15:0];
51         if (wr_set)   setpoint_reg <= pid_writedata[15:0];
52         if (wr_meas)  meas_reg     <= pid_writedata[15:0];
53         //if (wr_start) start <= 1;
54     end
55 end
56 //Read multiplexer
57 assign pid_readdata = (pid_address == 3'b110) ? {16'b0, pid_out} :
58                       (pid_address == 3'b111) ? {31'b0, done} :
59                        32'b0;
60
61 endmodule
62
```

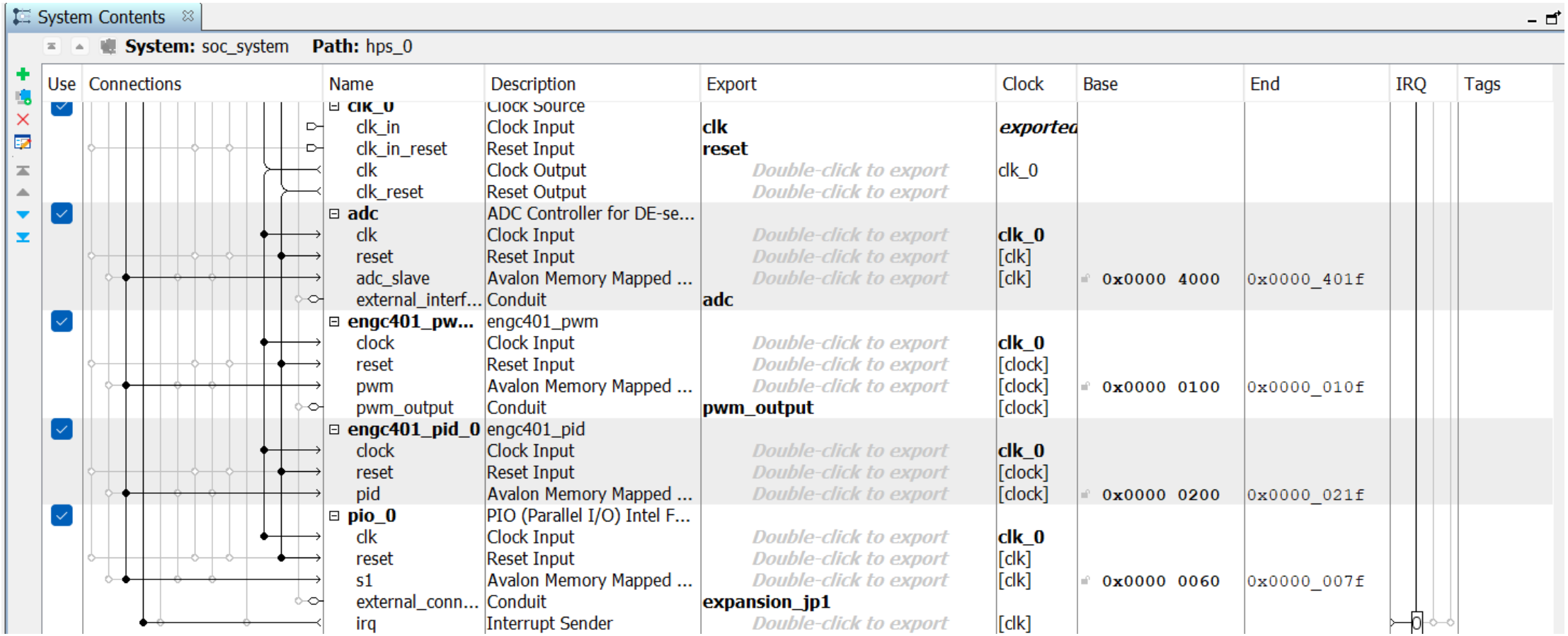
Fulfills requirement R.2: Design custom PID IP core wrapped in Avalon bus

PID IP Core



Fulfills requirement R.2: Design custom PID IP core

Hardware Accelerator Connections

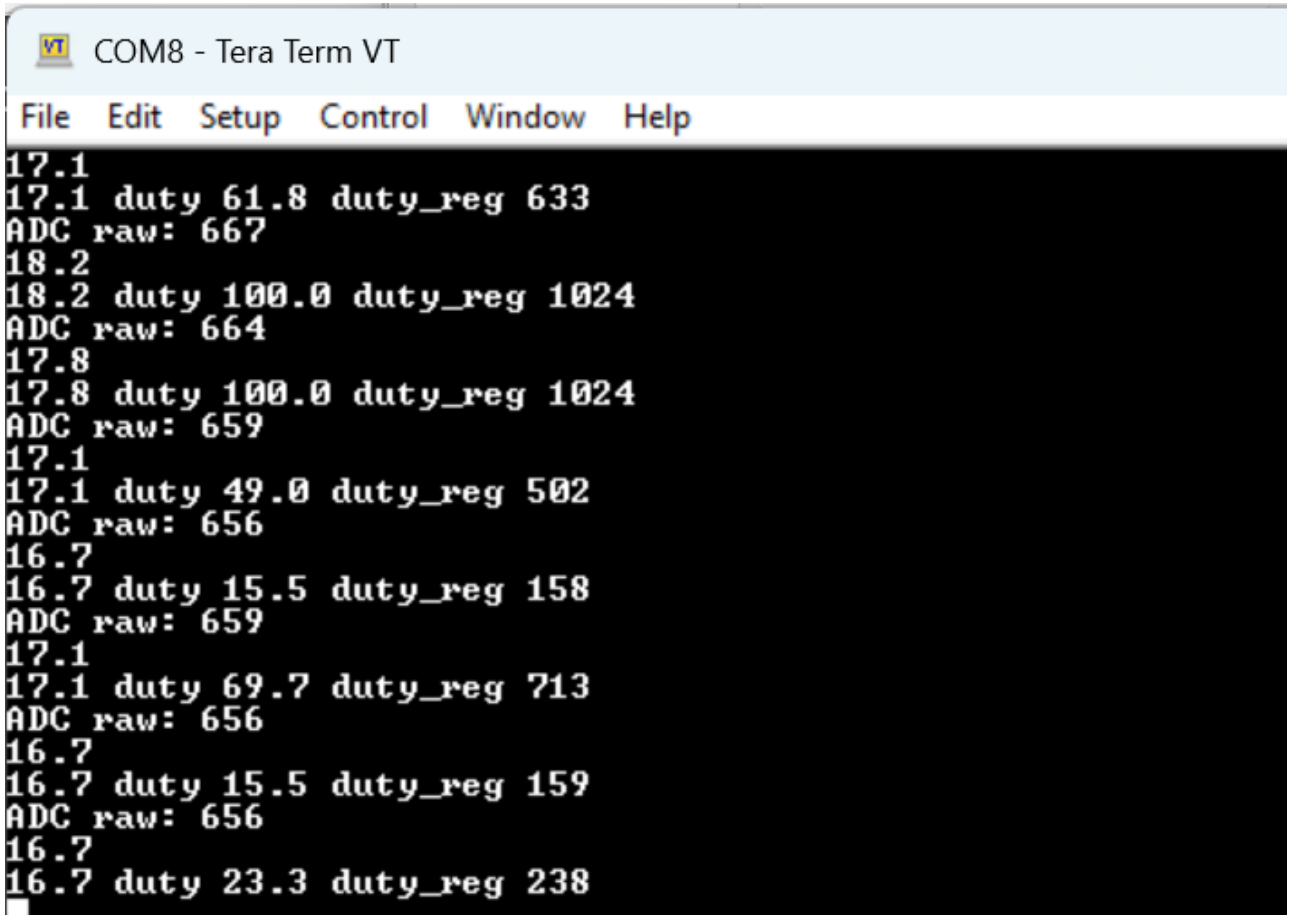


Fulfills requirement R.1: System must contain ADC, PWM, and PID IP cores

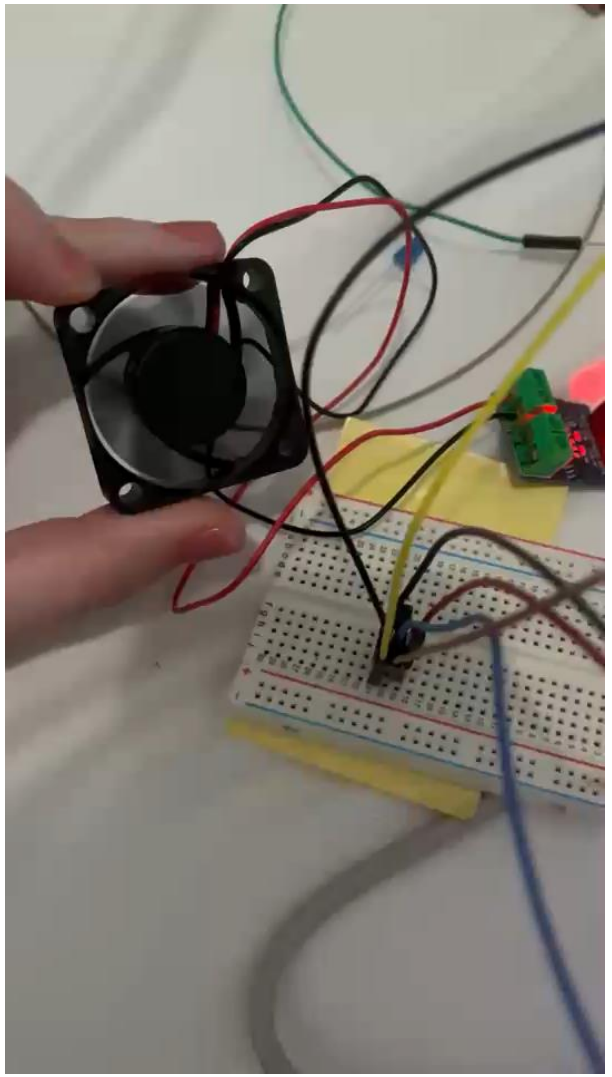
Software Connections

- Software writes ADC value and reference temp to PID IP core
- PID hardware computes the control output
- CPU waits until the PID calculation is finished, then reads the result
- Result is scaled and written to the PWM duty cycle register
- Fan speed adjusts in real-time based on output

Final Results



Final Results



Summary

- By designing a custom computer system with:
  - ADC, PWM, and PID IP cores
  - A PID IP core wrapped in the Avalon bus
  - Implementing the IP cores in hardware through memory mapping
  - Building a circuit to drive a DC fan
- We successfully fulfilled all the system requirements

Acknowledgements

- We would like to thank Dr. Wang for teaching us the past 3 years at Liberty
- Along with giving us the tools to make our dreams of becoming computer engineers come true
- We would also like to thank our classmates for their curiosity and guidance

Questions?